Airbnb Success: Prediction of Review Activity and Unsupervised Segmentation

Divya Andem (divyaand@umich.edu) - Jordan Huang (jordanhu@umich.edu) - Sophia Settle (sosettle@umich.edu)

Introduction

Airbnb has emerged as one of the largest online travel marketplaces, generating $11.1 billion in revenue in 2024 and processing 491 million bookings. By combining the Inside Airbnb dataset with external location-based demographics, crime rates, and walkability indices, this project seeks to yield actionable insights for hosts seeking to improve ratings and occupancy and for Airbnb to identify what type of Airbnb listing they own, as well as what factors influence getting more reviews. Understanding why some listings achieve higher frequency of reviews has important implications for hosts aiming to optimize their offerings and for the platform as it allocates supply and expands into new markets. This project is motivated by both the dramatic recent growth of the short-term rental market and our team members' enjoyment of traveling and staying in Airbnb listings.

In this project, we employed a combination of tree-based and linear regression models to predict review frequency. In addition to the predictive modeling, we used unsupervised methods of PCA and KMeans to uncover Airbnb archetypes. While many different analyses of the Inside Airbnb dataset exist, none brought in location-based crime and population to enrich the dataset and enhance modeling outputs and interpretability.

For the supervised component, we modeled Airbnb listing demand using reviews_per_year as a continuous outcome and compared four regression approaches: Linear Regression, K-Nearest Neighbors, Random Forest, and XGBoost. The feature set combines listing characteristics with external neighborhood context—including population density, crime, walkability, and proximity to key locations. Among the models, XGBoost achieved the strongest performance, indicating the presence of meaningful non-linear relationships in the data. Feature importance and ablation results show that minimum stay requirements, property configuration, and location factors are the primary drivers of demand, while host reputation variables contribute comparatively little. Overall, the results suggest that operational constraints and neighborhood context play a larger role in listing performance than host-level characteristics.

The clustering results revealed four interpretable Airbnb archetypes differentiated primarily by property scale, minimum-stay strategy, demand intensity, and neighborhood environment. These segments include high-capacity group homes, standard short-term units with steady turnover, and two distinct low-turnover segments operating under monthly or limited-availability strategies. Notably, guest ratings were consistently high across clusters, indicating that performance differences are driven more by operational strategy and utilization patterns than by satisfaction. Overall, the unsupervised analysis provides a market segmentation framework that complements the predictive models and offers actionable insights into how listing configuration and neighborhood context shape Airbnb supply dynamics.

Related Works

There are several publicly available projects that leverage the Inside Airbnb datasets. These projects apply methods such as sentiment analysis and time-series forecasting/price modeling across different cities.

The Inside-AirBNB-Dataset project centers on sentiment analysis of guest reviews, network analysis of host–neighborhood relationships, and time-series forecasting of average price and availability for Seattle, New York, and San Francisco. Unlike the above project's focus on time-series forecasting for a few cities, we will perform supervised learning to identify factors associated with review frequency and employ unsupervised clustering to identify Airbnb archetypes. In addition, we are enriching our Inside Airbnb dataset with demographic, crime and walkability. Lastly, we will be focusing solely on California cities. The inclusion of multiple external datasets and emphasis on popularity rather than price or sentiment differentiates our approach.

The Airbnb Explorer project aims to build a predictive model for future rental prices. It motivates the work by noting the increase in Airbnb listings in New York City and uses regression models such as linear, lasso, ridge, random forest, and gradient boosting to predict prices, presenting the results in a dashboard for hosts. Our study is distinct in both scope and methodology, as it targets multiple California cities rather than New York City alone and predicts Airbnb reviews rather than prices.

The AIR-BNB-DASHBOARD project is a Tableau-based dashboard built from Inside Airbnb data to explore the rental market across different cities worldwide. This project emphasizes visualizations of the Inside Airbnb data, such as maps, histograms of prices, bar charts of room types, scatter plots for availability, and a word-cloud for reviews. These visualizations are combined into an interactive dashboard that allows users to filter by city, room type, and price range, providing insights into listing density, price distribution, and availability .While the Tableau dashboard is primarily descriptive and focuses on visual exploration of Airbnb listings, our project incorporates machine-learning. We engineered features, built supervised models to predict review frequency, performed unsupervised clustering, and used external contextual data sources. Our analysis will therefore provide predictive and diagnostic insights rather than static visualizations.

Data Sources

Inside Airbnb

The base dataset for this project came from Inside Airbnb Data, and included Airbnb listing data from different cities across California including: Los Angeles, San Francisco, San Diego, and Oakland. These datasets represent 69K Airbnb listings and include important fields such as listing location, amenities, host information, ratings, reviews, availability, and other descriptions. These datasets were downloaded from the Inside Airbnb website as CSV files and represent scraped Airbnb listings as of September 2025. Data from all cities were combined into a single dataframe by adding a source_city column.

Crime

For the crime data, a total crime count per GEO ID was brought in. The project brings in a total crime count instead of specific crime types, as the crime classifications vary by city dataset. To maintain a more equal view of crime, this project initially includes the total crime count in the raw dataset. For Los Angeles, crime data is available through both the city's open data portal and the Los Angeles County Sheriff's Department. For all other cities, crime data is sourced directly from their respective city portals: San Francisco, San Diego, and Oakland. The appropriate year (2023 in our case) can be selected using the query options provided in each portal's user interface, and the data can then be downloaded or exported as CSV files. All datasets include latitude and longitude, incident date and time, and incident type information.

Population

The population dataset can be downloaded from data.census.gov by selecting the appropriate table (B01003), selecting the required fields and filters in the user interface, and then exporting the data as a CSV file. For this analysis, we selected all California census tracts for 2023. This dataset includes census tract ID, population estimates, and margin of error.

Walkability

The walkability dataset, published by the EPA, includes a nationally standardized Walkability Index (NatWalkInd) for the United States. The data are provided at the census block group level. The dataset was downloaded as a CSV and represents the 2021 release of the index, the most recent available.

Data Preprocessing

To prepare our Airbnb listing data for joining with the crime, population, and walkability datasets, each listing was assigned a GEOID (census tract ID) and the area of the corresponding GEOID based on its latitude and longitude. To do this, census tract shapefiles containing polygon geometries were downloaded, and the GeoPandas library was used to determine which polygon each point (latitude, longitude) falls within.

This combined Airbnb dataset was then left-merged with population data to attach population estimates for each GEOID. Crime data was processed in a similar manner by assigning a GEOID to each incident based on its latitude and longitude. The total number of crime incidents per GEOID was then calculated. The Airbnb data was left-merged with the crime counts so that each listing is associated with a crime count for its corresponding census tract. The walkability dataset provides GEOID in scientific notation (e.g., 4.8113E+11 ~ 48113000000), which can lose digits and become inaccurate if used directly. To preserve precision, GEOID was reconstructed by combining STATEFP (2 digits), COUNTYFP (3 digits), and TRACTCE (6 digits), padding each field with leading zeroes as needed (e.g., 48 + 113 + 7825 → 48 + 113 + 007825 = 48113007825) to create an 11-digit code. Since walkability is

measured at the block group level, scores were grouped by GEOID (census tract level) and averaged. The processed walkability data was then right-merged with Airbnb listings to assign each listing a walkability score based on its GEOID. The raw dataset includes all fields from the Inside Airbnb listings, source_city, along with total population, total crime,e and median walkability scores at the GEOID level.

Feature Engineering

The raw Inside Airbnb dataset contains a mix of textual, categorical, and numerical fields that require transformation before analysis. Feature engineering was conducted to convert non-numeric information into structured quantitative variables, reduce noise, and construct domain-relevant measures that better capture listing quality, host behavior, and location characteristics.

We first examined object-type columns to extract usable numerical information. Several binary variables, including host_is_superhost and instant_bookable, were converted from text indicators ("t"/"f") into numeric 0/1 values. Next, percentage values stored as text, such as host_response_rate and host_acceptance_rate, were converted to floating-point numbers. These features capture important aspects of host responsiveness and reliability, which are strong indicators of listing professionalism and potential guest satisfaction.

Text information was transformed using both aggregation and encoding strategies. The amenities field, originally a text list, was used to create two domain-specific features: safety_amenities_count (e.g., smoke detectors, fire extinguishers) and convenience_amenities_count (e.g., Wi-Fi, kitchen, workspace). These counts provide a compact representation of listing quality and functionality while avoiding the high dimensionality that would result from one-hot encoding individual amenities. The room_type variable was one-hot encoded to create binary indicators for major categories (e.g., entire home/apartment, private room, shared room). This allows models to capture structural differences between listing types without imposing artificial ordinal relationships. To capture niche or differentiated properties, the property_type field was used to create an is_unique_stay indicator identifying uncommon accommodation types such as treehouses, yurts, or caves. Unique properties may attract different demand patterns and pricing dynamics compared to standard housing.

In addition to encoding text information, several numerical features were engineered to better represent relationships and reduce collinearity. For example, price_per_person (price / accommodates), capturing cost efficiency relative to capacity, host_years, derived from host_start_date, reviews_per_year, derived from first_review, were created to help demand intensity and listing performance over time. These normalized metrics help distinguish high-performing listings from those that simply reflect larger size or longer platform tenure. Geographic coordinates (latitude and longitude) were used to engineer accessibility features dist_km_to_downtown and dist_km_to_airport. These features capture proximity to key urban amenities and transportation infrastructure, which are known drivers of Airbnb demand and pricing. Distance-based measures provide a more interpretable representation of location than raw coordinates and reduce spatial noise in downstream analysis. Log-transformed versions of these features were added to help with our distance-based models later on.

Beyond the Airbnb dataset, population and crime statistics were standardized to density-based measures: population_per_sqmile, crime_per_sqmile. These density metrics provided less skewed distributions and better reflect the environmental conditions experienced by guests and hosts.

Several variables in the dataset contained missing values. A structured imputation strategy was applied, informed by the domain context and the underlying data distribution, to preserve information while minimizing bias. For structural housing attributes such as bathrooms, bedrooms, and beds, missing values were imputed with 0. These cases primarily corresponded to studio-style listings or to minimal configurations in which separate room counts were not applicable. Other key contextual variables requiring imputation included price, population, and crime metrics. For price, missing values were imputed using the median price within the same city, neighborhood, and room type. This approach preserves local market conditions and avoids introducing unrealistic pricing levels. Similarly, population values were imputed using median values at the neighborhood and city level to maintain geographic consistency. Host responsiveness variables (e.g., response rate and acceptance rate) were missing primarily for newly created listings with no interaction history. These fields were imputed with a value of –1 to explicitly indicate the absence of historical data, allowing models to distinguish between missing information and low performance. Lastly, for spatially dependent neighborhood characteristics, a proximity-based approach was used. A custom nearest-neighbor imputation method was implemented using latitude and longitude coordinates. This

function identified geographically similar listings and imputed values for walkability (NatWalkInd) and crime-related features based on nearby observations. Leveraging spatial proximity ensures that imputed values reflect realistic local conditions and preserve geographic patterns in the data. Overall, missing data handling combined rule-based, group-level, and spatial imputation methods to retain dataset completeness while maintaining meaningful structural and geographic relationships.

Some Airbnb listings did not yet have any reviews. Because review rating and review_per_year are key target variables for the supervised methods and a key feature for the unsupervised methods, a blanket filter of count of reviews > 0 was applied, which reduced our dataset to ~52K rows/listings.

Based on the feature engineering process and EDA, a substantial number of variables were removed prior to all modeling exercises. The objective was to retain features that were informative, interpretable, and appropriate for clustering while reducing noise and redundancy. The features dropped included: unique identifiers (e.g., host IDs) and text fields (e.g., listing descriptions, host biographies, neighborhood overviews). These were removed because they do not provide meaningful numerical structure for modeling and would introduce high dimensionality or sparsity if encoded. Variables related to data collection timing, calendar updates, or platform metadata were excluded as they do not reflect intrinsic listing characteristics. In other cases, there were multiple variables that captured similar or identical information, so redundant features were dropped ,and a single representative feature was retained (e.g., availability at different time horizons, min_nights, max_nights, multiple types of review ratings, host counts). These were removed to avoid multicollinearity, unfair weighting in models, and noise. Lastly, original variables that had already been transformed or encoded into engineered features (e.g., amenities lists, host_since, raw population, and crime_counts) were removed to prevent duplication as well. For room_type, one-hot encoded variables that one category was removed to avoid the dummy variable trap and reduce unnecessary dimensionality. Beyond these column drops, each model, depending on its purpose, drops a few additional columns and will be detailed in the appropriate methods section.

Part A: Supervised Learning

Additional Data Preprocessing

In addition to the text, identifier, duplicate, redundant, and already transformed or aggregated columns dropped during feature engineering, supervised learning models require the removal of variables that are leaky or contain future information. Availability fields were removed as they indicate future bookings. review_scores_rating was removed as it is directly tied to the reviewing process. host_is_superhost, host_response_rate, host_acceptance_rate, and host_response_time were removed because they are outcomes of sustained booking and review activity and serve as indirect proxies for the target variable. Features used in calculating the target variable were also removed. The target variable reviews_per_year was clipped at 100, as values above this threshold represent the top 0.1–0.5% of listings. This removes outliers corresponding to viral listings with abnormally high review counts that are not representative of typical Airbnb performance. Additionally, numeric features were scaled using StandardScaler for KNN, bringing them to a mean of 0 and a variance of 1. Feature scaling is important for distance-based methods such as KNN, because features with larger ranges can disproportionately influence distance calculations. Binary variables were not scaled, as they are already on a comparable scale and represent membership rather than magnitude. For tree-based models, no scaling was applied, as these algorithms are insensitive to feature ranges. Tree-based models split data based on feature thresholds rather than distances, so the absolute scale of a feature does not affect model decisions. The final dataset for the supervised learning models contained 52124 rows, and the feature space was as follows:

- Target = reviews_per_year
- Features = 'host_listings_count', 'host_has_profile_pic', 'host_identity_verified', 'bedrooms', 'minimum_nights', 'maximum_nights', 'instant_bookable', 'NatWalkInd', 'safety_amenities_count', 'convenience_amenities_count', 'room_type_Hotel room', 'room_type_Private room', 'room_type_Shared room', 'is_unique_stay', 'population_per_sqmile', 'crime_per_1000', 'host_years', 'price_per_person','log_dist_km_downtown', 'log_dist_km_airport', 'source_city_OAK', 'source_city_SD', 'source_city_SF', 'bathrooms_per_bedroom'

Methods

Rating scores in the Inside Airbnb dataset are highly concentrated near the upper end of the scale (typically between ~4.5 and 5.0), reflecting a well-known positivity/ceiling effect in online ratings. This limited variance reduces the

signal for supervised learning and makes it difficult for models to meaningfully separate listings based on rating outcomes. In our initial experiments, models predicting review_scores_rating performed poorly, with a maximum validation R^2 of approximately 0.13. For the remainder of the supervised analysis, we therefore focused on reviews_per_year as the target, since it serves as a practical proxy for listing popularity and demand intensity: listings that attract more bookings generally accumulate reviews at a higher rate.

Because reviews_per_year is a continuous outcome, the supervised task was framed as a regression problem. To evaluate a range of modeling approaches, four regression model families with different learning mechanisms were compared: linear regression, K-nearest neighbors (KNN), random forest, and XGBoost. These models represent a mix of linear, instance-based, and tree-based ensemble methods commonly used for structured tabular data. With additional time, the outcome could also be modeled as a classification problem (e.g., low/medium/high demand) by binning reviews_per_year; however, regression was preferred to preserve the full variability of the target.

Linear Regression, a parametric model, was used as a baseline due to its simplicity and interpretability. It assumes linear relationships between features and the target, making it easy to understand and reason about. However, it performed poorly, suggesting non-linear relationships between features and the target. The poor performance was initially hypothesized to be driven by the extreme left skewness in the target variable (review ratings). However, even after switching the target to reviews per year applying a log transformation, the linear models did not improve, suggesting the presence of a nonlinear relationship. For consistency in the results between the linear regression models and the remaining models, the non-transformed reviews_per_year was kept as the target variable.

K-Nearest Neighbors (KNN) was explored as an instance-based, non-parametric method that makes predictions based on the similarity of a listing to its nearest neighbors in feature space. It requires no assumptions about the underlying data distribution and is easy to interpret. KNN did not perform well overall, likely due to the curse of dimensionality, as distance metrics become less meaningful in high-dimensional spaces.

Random Forest is an ensemble tree-based bagging method that builds multiple decision trees on random subsets of data and features, reducing variance through averaging. It achieved a meaningful improvement over KNN, demonstrating that non-linear interactions between features are important for this task.

XGBoost is an ensemble tree-based boosting method where trees are built sequentially, with each tree correcting the residual errors of the previous one. This iterative error correction, combined with built-in regularization, makes it well-suited for structured tabular data. It achieved the best performance across all metrics and was selected as our final model.

Hyperparameter Tuning

Hyperparameter tuning was performed using GridSearchCV with 5-fold cross-validation for all models, optimizing for MAE as the primary metric. For KNN, values of n_neighbors between 11 and 101 were explored, along with two weighting mechanisms: uniform and distance. Distance-based weighting resulted in severe overfitting, with train R^2 of 0.99 versus validation R^2 of 0.30. Uniform weighting was therefore selected, which significantly reduced the train-validation gap. The final cross-validation results are reported below in Table A-1. For Random Forest, the search grid explored the number of trees, tree depth (10–20), minimum samples required to split a node, minimum samples per leaf (20–500), and the fraction of features considered at each split. A fixed subsample rate of 0.8 was used. All combinations were evaluated using GridSearchCV with 5-fold cross-validation, and the results are reported in Table A-1. For XGBoost, initial tuning used deep trees (max_depth of 15–20) with small leaf sizes (min_child_weight of 5–20) and no regularization. The best model from this grid exhibited severe overfitting — a train R^2 of 0.968 versus a validation R^2 of 0.472, and a train MAE of 1.51 versus a validation MAE of 6.50. To address this, the parameter grid was revised to use shallower trees (max_depth of 5–10), enforce larger leaf sizes (min_child_weight of 50–200), and introduce L1 and L2 regularization. The revised best model significantly reduced overfitting, train R^2 dropped from 0.968 to 0.691, while validation R^2 remained stable at 0.452. Although validation MAE increased slightly to 6.77, this tradeoff was accepted in favor of a more generalizable model. Overall, this workflow allowed systematic comparison across diverse model families, and XGBoost was selected as the final model because it provided the best balance of predictive performance and generalizability.

Results

We evaluate all models using three metrics: Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and R^2. MAE measures average prediction error in the original unit (reviews per year), making it directly interpretable. RMSE penalizes large errors more heavily than MAE, which is useful for identifying models that produce extreme mispredictions. R^2 measures the proportion of variance explained by the model. Table A-1 reports 5-fold cross-validation results for all models. Mean and standard deviation are reported across folds.

Model	R^2	MAE	RMSE
Linear Regression	-0.8086 ± 1.7610	9.8954 ± 0.0938	18.6849 ± 7.4108
KNN	0.2731 ± 0.0090	8.1643 ± 0.0299	11.9585 ± 0.0641
Random Forest	0.3855 ± 0.0053	7.2832 ± 0.0462	10.9955 ± 0.0998
XGBoost	0.4521 ± 0.0047	6.7735 ± 0.0583	10.3824 ± 0.0915

Table A-1

In-Depth Evaluation of Best Model – XGBoost

Feature Importance and Ablation Analysis

Top features for the XGBoost model were identified as: minimum_nights (0.191), bathrooms_per_bedroom (0.074), source_city_SD (0.066), room_type_Private room (0.063), and source_city_OAK (0.061). We performed ablation analysis by cumulatively removing the top features in order of importance and measuring the impact on validation performance. Ablation Analysis results are shown below in Table A-2.

Removed Count	MAE	R^2
0 (baseline)	6.7735 ± 0.0583	0.4521 ± 0.0047
1	7.6987 ± 0.0680	0.3426 ± 0.0086
2	7.8658 ± 0.0734	0.3216 ± 0.0067
3	7.8848 ± 0.0701	0.3196 ± 0.0060
4	8.0020 ± 0.0684	0.3041 ± 0.0055
5	8.0062 ± 0.0677	0.3035 ± 0.0056

Table A-2

With all features intact, the model achieves an MAE of 6.77 and an R^2 of 0.45. Removing just the top feature (minimum_nights) causes MAE to increase by 13.7% (6.77 → 7.70) and R^2 to drop by 24.2% (0.4521 → 0.3426), confirming that minimum_nights is by far the most influential feature. As features are removed cumulatively, performance continues to degrade, with MAE reaching 8.01 and R^2 falling to 0.30 after all five features are removed — a total degradation of 18.3% in MAE and 32.9% in R^2. Notably, the largest single drop occurs at the removal of the first feature, with diminishing impact from each subsequent removal.

We also performed a reverse ablation by cumulatively removing the bottom 5 least important features. Even after removing all five, MAE remains at 6.81 ± 0.07 and R^2 at 0.445 ± 0.004 , a negligible change compared to the baseline (MAE - 0.6% and R^2 - 1.6%). This confirms that the bottom features contribute little to model performance and could be removed to simplify the model without a meaningful loss in predictive accuracy.

Sensitivity Analysis

We perform sensitivity analysis on the best-performing model (XGBoost) by varying two key hyperparameters, max_depth and min_child_weight, while holding all other parameters fixed at their optimal values.

Max Depth	Train MAE	Val MAE	Train R^2	Val R^2
5	6.6995 ± 0.0102	7.1043 ± 0.0629	0.4782 ± 0.0014	0.4101 ± 0.0039
8	5.7290 ± 0.0125	6.8630 ± 0.0631	0.6136 ± 0.0019	0.4407 ± 0.0042
10	5.1100 ± 0.0172	6.7735 ± 0.0583	0.6910 ± 0.0021	0.4521 ± 0.0047

Table A-3

Seen in Table A-3, as max depth increases from 5 to 10, validation MAE improves from 7.10 to 6.77 and validation R^2 increases from 0.41 to 0.45, indicating that the model benefits from added complexity and reduced bias. However, the train–validation gap widens noticeably at depth 10, with train R^2 reaching 0.691 compared to validation R^2 of 0.452, signaling increasing overfitting. The model is clearly sensitive to this parameter: while deeper trees continue to improve validation performance up to this point, they also increase variance. Although overfitting has not yet degraded validation metrics, the expanding gap suggests that increasing depth beyond 10 would likely amplify overfitting without yielding meaningful additional improvements in validation performance.

Min Child Weight	Train MAE	Val MAE	Train R^2	Val R^2
50	5.1100 ± 0.0172	6.7735 ± 0.0583	0.6910 ± 0.0021	0.4521 ± 0.0047
100	5.6576 ± 0.0123	6.8657 ± 0.0630	0.6223 ± 0.0015	0.4410 ± 0.0050
200	6.1390 ± 0.0122	6.9622 ± 0.0624	0.5563 ± 0.0009	0.4277 ± 0.0044

Table A-4

Results from min_child_weight are shown in Table A-4. Min child weight controls the minimum number of samples required to split a node. Higher values force the tree to generalize more by preventing splits on small groups of samples. However, in our results, increasing min child weight from 50 to 200 worsens validation MAE from 6.77 to 6.96, suggesting the model is being over-regularized, and useful patterns in the data are being ignored. A value of 50 allows enough granularity and performs best in our case.

Learning Curve

Figure A-1 suggests that the XGBoost regression model is generally stable, as performance is not highly sensitive to training set size. The gap between training and validation curves decreases slightly, indicating that additional data may improve generalization and reduce overfitting. However, the near-plateaus of both curves suggest that meaningful performance gains are more likely to come from additional feature variability than from increased sample size.

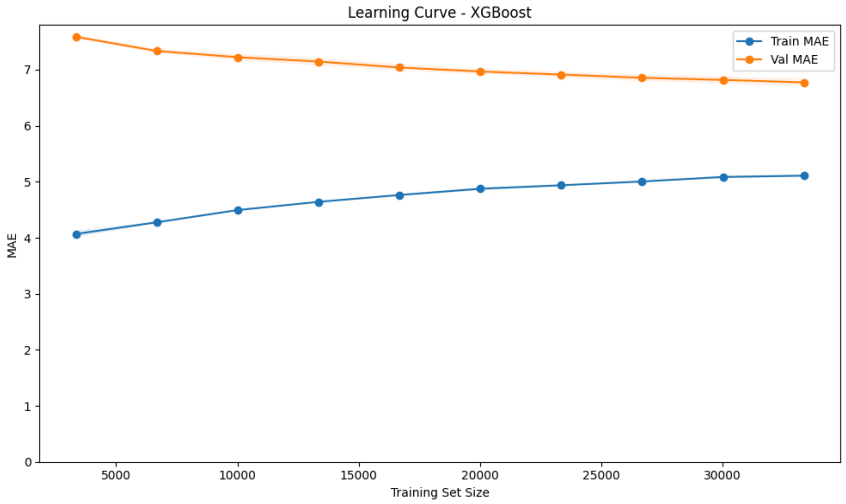


Figure A-1

Additional Tradeoffs

As max depth increases from 5 to 10, validation MAE improves from 7.10 to 6.77, and validation R^2 increases from 0.41 to 0.45, indicating that the model benefits from added complexity and reduced bias. However, deeper trees substantially widen the train–validation gap, signaling severe overfitting and increased computational cost. Shallower trees were therefore selected, accepting a marginal loss in validation performance in favor of a more generalizable and stable model.

The learning rate presents a tradeoff between training speed and model stability: higher values allow faster convergence with fewer trees but increase the risk of overfitting, while lower values promote more stable learning and better generalization at the cost of longer training time and potential underfitting if the number of trees is insufficient.

Additionally, dropping features such as text descriptions and images may cause the model to lose experiential signals. While this simplification reduces model complexity and training time, it comes at the cost of predictive performance, as the model cannot capture these nuanced aspects of listings. Our best-performing model achieved a validation R^2 of only around 0.45.

Failure Analysis

Missing/Mis-imputed Features — *Listing ID 1076824435567752150*
Actual: 67.79 | Predicted: 2.25 | Error: 65.54
This listing had missing bathroom data imputed as 0, resulting in a bathrooms_per_bedroom value of 0, while the other features appeared normal. The model likely interpreted this as a low-quality listing and therefore severely underpredicted the target. This failure highlights the risk of overly simplistic imputation strategies that treat missing values as true zeros. Future improvements could include using more informed imputation methods or adding a binary indicator feature to explicitly capture missingness, preventing the model from interpreting imputed zeros as genuine observed data.

Latent Listing Quality — *Listing ID 873169882418157150*
Actual: 86.72 | Predicted: 17.6 | Error: 69.12
Despite reasonable pricing, the model predicted only 17 reviews per year, whereas the true value was 86. The host likely benefited from high-quality photos or a well-optimized listing profile — factors not captured in our feature set. This represents a latent listing-quality failure, in which important signals driving guest attraction are absent from the model. Adding features such as photo count or quality scores could address this.

Latent Experiential Signals — *Listing ID 833485670916105438*
Actual: 72.36 | Predicted: 13.13 | Error: 59.23
The listing had a low safety_amenities_count, which the model used as a proxy for quality, leading to underprediction. However, the listing still attracted many guests, likely due to intangible factors such as ambiance, spaciousness, or overall guest experience that are not encoded in structured features. Since the model does not incorporate textual or semantic features from listing descriptions, these experiential signals were invisible to it. Including NLP-derived features from listing descriptions could help capture such qualities.

Multi-Unit Property Demand — *Listing ID 1043748646220165881*
Actual: 100.00 | Predicted: 26.8 | Error: 73.20
The model underpredicted for a multi-unit property, estimating only 26 reviews against a true value of 100. Multi-unit properties typically receive more bookings and reviews than single-unit listings, but our dataset lacks a feature distinguishing between the two. The model, therefore underestimated demand based solely on amenity and listing-level features. Adding a multi-unit indicator feature would allow the model to account for this systematic difference.

New Listings with No Review History — *Listing ID: 1201432814078237120*
Actual: 1.49 | Predicted: 35.33 | Error: 73.20
This failure occurred for a new listing with no booking history. Since listing_age_years was removed due to leakage, the model relied solely on structural features (e.g., amenities, bedrooms, minimum_nights) and predicted a high review count for a newly created listing with attractive characteristics. This represents a classic cold start problem, as the model cannot distinguish between new and established listings. A potential improvement would be to introduce a non-leaky proxy for listing maturity or apply a separate strategy for new listings.

Part B: Unsupervised Learning

Additional Data Preprocessing

For the unsupervised portion of the project, additional data preprocessing was necessary to address issues arising in the clustering phase of part B. For example, latitude and longitude were dropped to avoid location-based clustering overpowering other information about the listings. At the first pass of clustering, when latitude and longitude were included, the clustering model was returning just a Northern California and a Southern California cluster. To achieve more meaningful results, these columns were dropped. After these additional data cleaning steps, 1% of rows were dropped due to containing NaNs (review_score_cleanliness contained a small amount of NaNs, this column was

dropped from part A due to being tied to the target variable). This resulted in a slightly lower final row count than part A of the project. The final dataset for the unsupervised portion had 52,117 rows and a final feature space of: 'host_response_rate', 'host_acceptance_rate', 'host_is_superhost', 'host_listings_count', 'accommodates', 'bathrooms', 'bedrooms', 'beds', 'price', 'minimum_nights', 'maximum_nights', 'availability_365', 'review_scores_rating', 'review_scores_cleanliness', 'instant_bookable', 'reviews_per_month', 'NatWalkInd', 'host_response_time_ord', 'safety_amenities_count', 'convenience_amenities_count','room_type_Hotel room', 'room_type_Private room', 'room_type_Shared room', 'is_unique_stay', 'population_per_sqmile', 'host_years', 'crime_per_sqmile', 'log_dist_km_downtown', 'log_dist_km_airport', 'listing_age_years', 'reviews_per_year'.

Prior to PCA and Kmeans Clustering, additional transformations were applied to improve feature comparability and reduce the influence of extreme values. All transformations were performed on a working dataset (df_scaled), while the original dataset (df) was retained to support post-clustering interpretation and visualization. Several variables exhibited strong right-skewed distributions, including price, population_per_sqmile, and crime_per_sqmile. These features were log-transformed to reduce the impact of extreme outliers, stabilize variance, and improve the ability of distance-based algorithms to identify meaningful structure. Following the transformation, feature scaling was performed using StandardScaler, which centers each variable at zero mean and unit variance. Standardization is critical for clustering because distance-based methods (such as K-means and PCA) are sensitive to differences in scale; without scaling, variables with larger numeric ranges would dominate the distance calculations. Binary one-hot-encoded variables were excluded from scaling because they already exist on a consistent 0/1 scale and represent categorical membership rather than magnitude. Standardization would distort their interpretation and reduce their meaningful contribution to distance calculations. This standardized feature matrix was then the basis for two unsupervised learning approaches: PCA and K-means Clustering.

Methods

The two primary unsupervised methods that were chosen for part B were PCA and KMeans. PCA was selected for better visualization, to eliminate collinear variables (e.g., capacity measures, demand indicators, and neighborhood characteristics), and to help the distance-based clustering method K-means to perform more effectively (orthogonal and low-dimensional features). By transforming the standardized variables into a smaller set of uncorrelated components, PCA improves cluster stability while preserving the dominant structure of the data. For the second unsupervised method, K-means was selected because it is a computationally efficient, widely used distance-based clustering algorithm that performs well on standardized, continuous feature spaces and provides clear, interpretable partitions suitable for segmenting Airbnb listings into distinct market archetypes.

First, PCA was performed on the final feature space and the cumulative explained variance was plotted against the number of principal components (Appendix B-1) to evaluate how much information was retained as dimensionality increased. In addition to variance analysis, component loadings were examined to interpret the underlying meaning of the principal components and ensure that the transformed feature space remained interpretable. The first 5 principal components capture interpretable dimensions such as property scale, urban accessibility, and listing performance (Table B-1).

PC	Interpretation	Top Positive Loadings	Top Negative Loadings
PC1	Property Scale	beds (0.40) accommodates (0.37) bathrooms (0.37)	population_per_sqmile (-0.18) listing_age_years (-0.16) crime_per_sqmile (-0.15)
PC2	Urban Accessibility	crime_per_sqmile (0.36) host_response_rate (0.34) population_per_sqmile (0.34)	log_dist_km_downtown (-0.42) log_dist_km_airport (-0.25) bedrooms (-0.12)
PC3	Demand & Performance	reviews_per_year (0.44) reviews_per_month (0.43) review_scores_cleanliness (0.29)	bedrooms (-0.25) accommodates (-0.23) crime_per_sqmile (-0.22)
PC4	Location vs. Quality & Experience	log_dist_km_airport (0.25) log_dist_km_downtown (0.25) host_response_rate (0.22)	review_scores_rating (-0.42) review_scores_cleanliness (-0.40) host_years (-0.30)

PC5	High Turnover vs. Service Quality	reviews_per_month (0.40) reviews_per_year (0.39) accommodates (0.17)	host_response_time_ord (-0.42) review_scores_rating (-0.35) review_scores_cleanliness (-0.35)

Table B-1

After interpreting the components, an initial K-Means clustering was performed utilizing the first 13 PCs, which retained 80% of the cumulative variance in the initial feature space. This 80% threshold was chosen to balance dimensionality with information retention. Using WCSS and an "elbow plot" (Appendix B-2), optimal K=4 was selected. This initial K=4 clustering was then plotted (Figure B-2), and showed fairly distinct clusters.

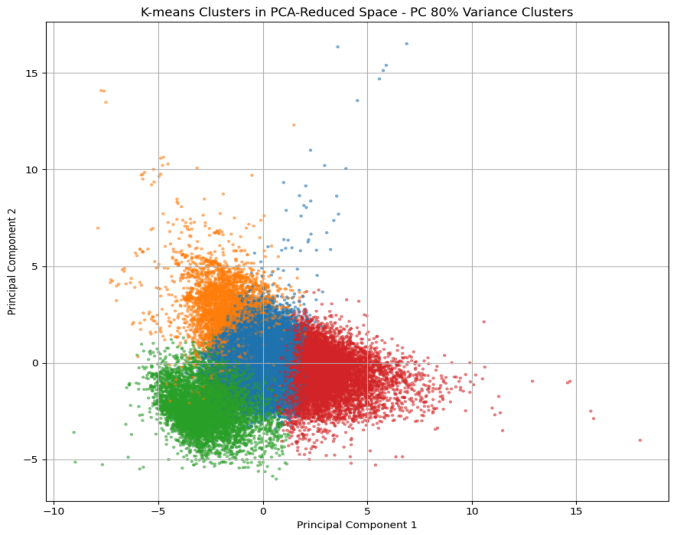


Figure B–2

Next, the initial selection of K=4 using the 13-PC feature space was evaluated across a range of K values using the silhouette score, which indicated K = 4 and K = 5 as optimal with extremely similar silhouette scores (Appendix B-3). When the K =5 version was plotted it was obvious that K=5 created a small, non-distinct 5th cluster.

To improve the clustering further and identify the principal components most relevant for cluster differentiation, F-statistics were computed for each component based on the initial K=4 80% Variance / 13-PCs clusters. The results showed a clear ranking of component importance: PC1 > PC2 > PC3 > PC5 > PC4. After PC4, there was a substantial drop in F-statistic values, indicating that additional PCs contributed little to cluster separation. Based on this analysis, the feature space was reduced to PC1–PC5.. This targeted dimensionality reduction prioritizes components that improve segmentation rather than those that simply capture overall variance. K-means clustering was then re-run using the reduced PCA representation and evaluated across multiple values of K using the silhouette score (Appendix B-4). The refined feature space silhouette score analysis space supported a 3 or 4 cluster solution. The new visualization of the results showed clearer and more distinct cluster separation (Figure B-3).

This iterative process demonstrates that clustering performance is sensitive not only to the choice of K, but also to the underlying feature representation. Selecting principal components based on their ability to differentiate clusters resulted in a more stable and interpretable segmentation of Airbnb listings.

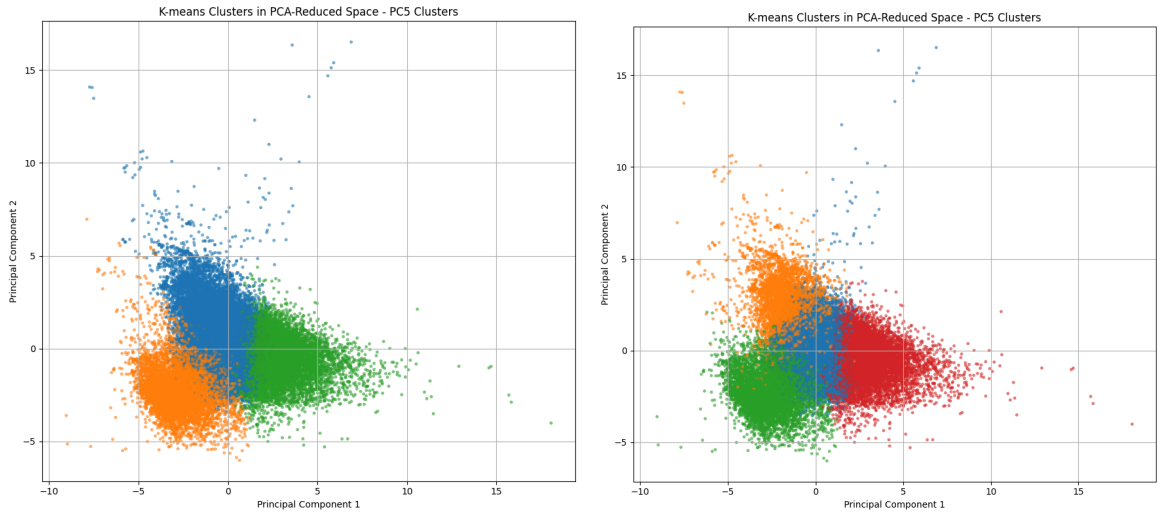


Figure B-3

Evaluation + Sensitivity Analysis

To evaluate clustering performance across the different models, three criteria were used: the silhouette score (to assess cluster separation), within-cluster sum of squares (WCSS; to assess compactness), and normalized entropy (to assess how balanced the cluster size distribution was). Together, these metrics provide a more complete view of clustering quality than any single measure. The results are seen below in Table B-2.

Model	Silhouette Score	WCSS (Inertia)	Normalized Entropy
80% PCs (k=4)	0.186	784,734	0.839
80% PCs (k=5)	0.189	757,840	0.733
Top 5 PCs (k=3)	0.278	420,112	0.820
Top 5 PCs (k=4)	0.275	363,042	0.875

Table B-2

Results indicated that the 80% variance 13-PC representation with K = 4 performed the weakest overall, with the lowest silhouette score and the highest WCSS. This suggests that retaining many principal components preserved substantial noise and irrelevant variation, leading to overlapping clusters and limited separability. However, increasing the number of clusters to K = 5 in the 80% variance 13-PC space did not meaningfully improve silhouette score and instead reduced normalized entropy, indicating more uneven cluster sizes and suggesting that the additional cluster captured noise rather than meaningful structure.

In contrast, restricting the feature space to the top five principal components substantially improved cluster quality. Both the K = 3 and K = 4 models in the Top-5 PCA space achieved significantly higher silhouette scores and far lower WCSS, indicating tighter cluster structure and improved separation. While K = 3 configuration produced the highest silhouette value, the K = 4 model achieved nearly equivalent separation while improving WWCS and producing a more balanced cluster distribution, which supports its suitability for the specific goals of this Airbnb project to provide informative segmentation.

Overall, the final model selected was the Top-5 principal component feature space with K = 4. This configuration offered the best balance between separation, compactness, and cluster balance, while also providing greater segmentation granularity than the K = 3 alternative with minimal loss in silhouette performance.

To assess the stability between the two K=4 feature spaces, a sensitivity analysis of cluster movement was performed. Specifically, the analysis examined how decreasing the dimensionality affected cluster labels. The two models showed that 94.9% of listings remained in the same cluster, with only 5.1% reassigned. The high level of agreement suggests that the four-cluster structure is fairly stable and not highly sensitive to small changes in

dimensionality. This analysis, paired with our metric evaluation still points to the Top 5 PC K = 4 solution being the best model for clustering. The biggest movement was out of cluster 0 (center), which is also the largest cluster.

Figure B-4 illustrates cluster stability visually. Listings that remained in the same cluster are shown in grey, while reassigned observations are highlighted in red. The limited number of changes further confirms the robustness of the selected representation.

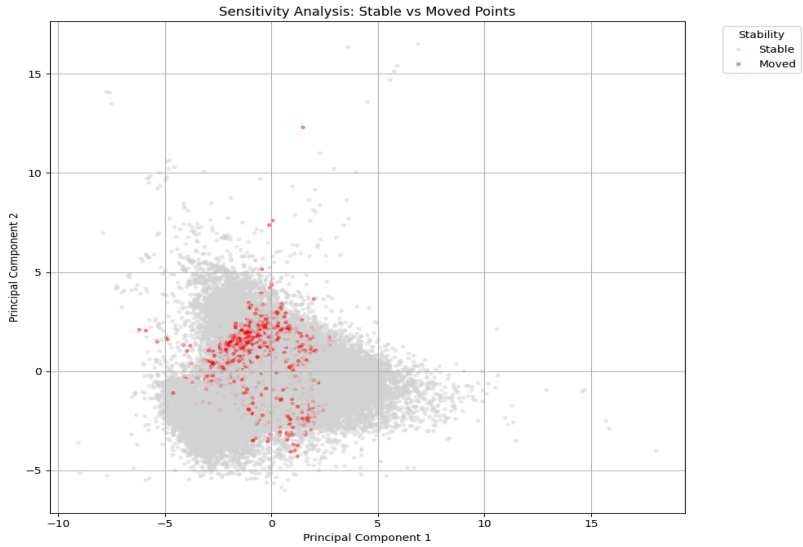


Figure B-4

Interpreting the Results

Figure B-5 shows a standardized comparison of cluster profiles based on the average of the 15 most differentiating features. To highlight relative differences across segments, feature means were converted to z-scores calculated across clusters, allowing each cluster's strengths and weaknesses to be interpreted in relation to the overall distribution. This standardized view emphasizes which attributes are comparatively high or low within each segment while minimizing the influence of scale differences across variables. The figure provides a concise visual summary of the characteristics that most clearly distinguish the four Airbnb clusters.

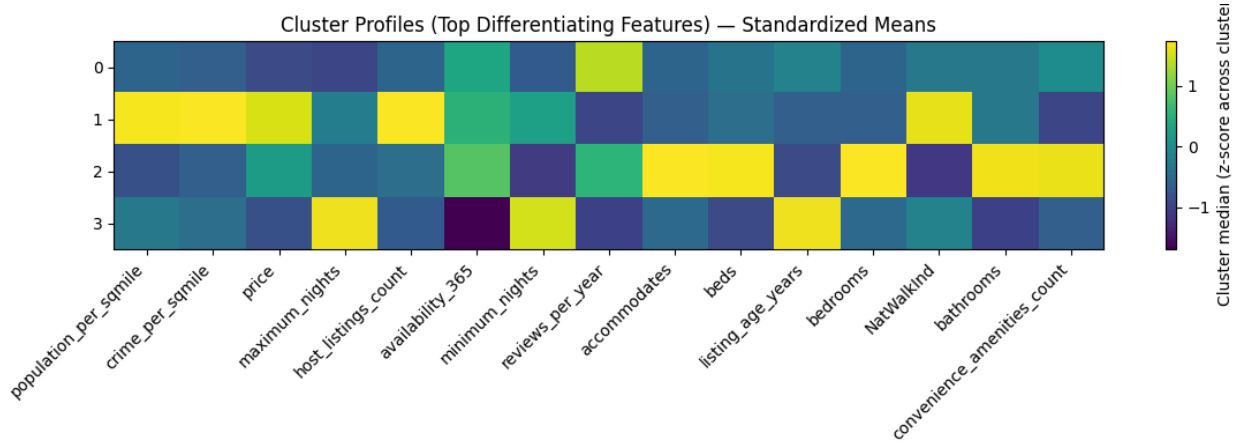


Figure B-5

The four-cluster solution reveals distinct segments of Airbnb listings differentiated by property scale, stay strategy (short vs monthly), demand intensity, and neighborhood density/crime context.

Cluster 0 represents standard, high-quality short-stay entire-home listings oriented around couples or small groups. Median listings accommodate 2 guests with 1 bed/1 bath and a 3-night minimum stay. They show strong review activity (1.25 reviews/month, 8.9 reviews/year) and excellent guest satisfaction (median rating 4.91, cleanliness

4.90). These listings tend to sit in moderate-density neighborhoods (median population per sq mi 10.4k) with lower crime relative to the highest-density cluster, suggesting a "reliable, mainstream" inventory segment.

Cluster 1 captures small, monthly-stay listings that function more like mid-term housing than classic short-term rentals. These are also typically 2-person, 1-bed/1-bath units, but they require a 30-night minimum stay and have much lower demand signals (0.28 reviews/month, 2.19 reviews/year). They maintain solid ratings (4.77) and cleanliness (4.76), but are the lowest median of all clusters. Their defining characteristics are low turnover and location in the highest-crime environments of all clusters (median population per sq mi 26.6k, crime per sq mi 4509.8). This segment likely reflects longer-occupancy usage patterns where review volume is naturally suppressed.

Cluster 2 represents large, group-oriented homes and is the clearest "capacity" segment. Median listings accommodate 8 guests with ~3 bedrooms, ~2.5 bathrooms, and ~4 beds, with a 2-night minimum stay. They also have the highest price point while still sustaining strong demand (1.22 reviews/month, 7.7 reviews/year) and very high satisfaction (rating 4.92, cleanliness 4.89). These listings skew toward lower-density, lower-crime neighborhoods (population per sq mi 8.3k, crime per sq mi 268.9), consistent with spacious housing stock suited for families/groups.

Cluster 3 consists of sparse-availability, monthly-stay listings with low review activity, but importantly they are not low-rated. These listings typically accommodate 2 guests and require a 30-night minimum stay, yet show near-zero availability and the lowest demand (0.16 reviews/month, 1.73 reviews/year). Despite low turnover, they maintain excellent ratings (4.91, cleanliness 4.89). They also stand out as the oldest listings by far (median listing age 6.46 years). Neighborhood context is moderate density (11.4k) with moderate crime (490), suggesting this segment may reflect legacy or consistently-occupied inventory where availability and reviews stay low because the unit is effectively "taken" most of the year.

Overall, the segmentation highlights interpretable differences across property scale (2-person vs 8-person homes), operational strategy (short stays vs 30-night minimums), market activity (review volume as a proxy for turnover/demand), and neighborhood context (density and crime). Notably, ratings remain high across clusters, with the main differentiation coming from stay rules, availability, and review activity rather than guest satisfaction – which reinforced the discoveries during feature importance analysis in Part A.

Discussion

One of the primary challenges in this project involved data integration and preparation across multiple sources. Assigning consistent geographic identifiers (GEOIDs) to Airbnb listings required careful spatial processing, and aligning crime data across cities was particularly complex due to differences in reporting formats and data availability. In addition, the Inside Airbnb dataset contained many highly correlated, redundant, and sparse variables, while the external population and crime features introduced substantial skew. Addressing these issues required iterative feature selection, transformation, and validation to ensure a stable and interpretable modeling dataset.

Over the course of this project, there were many trade-offs made regarding the data. To incorporate reliable crime data, the analysis was limited to a subset of California cities with consistent public data. While this decision reduced the overall sample size and geographic coverage, it enabled the inclusion of richer contextual features that improved the interpretability of both the supervised and unsupervised analyses. This reflects a broader tradeoff between dataset breadth and feature depth that shaped the overall project design.

Several challenges and insights emerged during the supervised learning portion of the project. One of the most notable findings was the limited usefulness of review ratings as a prediction target. Rating scores in the Inside Airbnb dataset are highly concentrated near the upper end of the scale (typically between 4.5 and 5.0), reflecting a ceiling effect common in online review platforms. This limited variance provided little predictive signal, and initial models achieved a maximum validation R^2 of only 0.13. As a result, the analysis shifted to reviews_per_year as the primary outcome, which more effectively captures variation in listing demand and substantially improved model performance.

Another surprising result was the magnitude of overfitting observed in the initial XGBoost model, where training R^2 reached 0.968 while validation R^2 was only 0.472. This large gap highlighted the sensitivity of boosting methods to model complexity and the importance of controlling depth and regularization. After tuning these parameters, the

train–validation gap narrowed, and the final model achieved a more balanced bias–variance tradeoff. Feature ablation analysis also revealed that performance was highly dependent on a small number of variables, particularly minimum_nights, indicating that a few operational constraints play a dominant role in predicting demand.

A key challenge throughout Part A was identifying and removing features that could introduce leakage or act as indirect proxies for the target. Because many Airbnb variables reflect outcomes of booking activity or platform dynamics, careful feature selection was necessary to avoid artificially inflated performance and ensure realistic generalization. In addition, the weak performance of linear regression confirmed that relationships between listing characteristics and demand are highly nonlinear, motivating the use of tree-based ensemble methods.

With additional time and resources, the supervised analysis could be extended in several ways. Alternative dimensionality-reduction approaches, such as partial least squares, could be explored to address multicollinearity while preserving predictive structure. Expanding the dataset beyond California would improve geographic generalizability, and incorporating additional signals—such as textual descriptions, image features, or seasonal demand patterns—could help capture latent aspects of listing quality that are not reflected in structured variables.

One of the most surprising parts of the unsupervised learning analysis was the model's sensitivity to slight changes in feature spaces. Small changes to the features/columns included before the PCA dimensionality reduction, as well as the number of PCs retained in the feature space for clustering, had a huge impact on the models. This also reflected in the main challenges for the unsupervised learning part: high dimensionality, correlated features, and early results that were driven primarily by geographic variation rather than listing characteristics. These issues were addressed through feature reduction, PCA refinement using F-statistics, and iterative evaluation with silhouette analysis and stability testing.

With additional time, the analysis could be extended by testing alternative clustering methods (e.g., Gaussian Mixture Models or hierarchical clustering), incorporating additional contextual features such as seasonality or amenities, and validating the segments against business outcomes such as occupancy or revenue to assess their practical value. In addition, adding more geo-based amenity features, like the number of parks or tourist attractions near the listing could be beneficial to the clustering exercise.

Ethical Considerations

Ethical considerations for this analysis primarily relate to data quality, privacy, and the responsible interpretation of location-based insights. One concern involves the reliability of publicly reported crime data. Differences in reporting practices across cities, changes in methodology over time, and reported crime data errors can lead to inconsistencies that may affect the accuracy and comparability of crime measures. In addition, the Inside Airbnb dataset is collected through web scraping rather than official platform data, which may result in incomplete coverage, outdated records, or measurement error.

Privacy is also an important consideration. Although the data used are publicly available and aggregated at the listing level, care should be taken to avoid drawing conclusions about individual hosts or specific properties. Results should be presented at an aggregate level to minimize the risk of unintended identification or misuse.

A further concern relates to the use of location-based variables such as crime rates and population density. Without appropriate context, these features may reinforce negative perceptions or stigmatize certain neighborhoods. To mitigate this risk, findings should be framed as descriptive market patterns rather than value judgments about specific areas.

Overall, these considerations highlight the importance of transparency about data limitations, cautious interpretation of results, and responsible communication to ensure that the analysis is both accurate and ethically applied.

Statement of Work

Below we have listed who was lead on certain components, but each team member touched almost every aspect of the project in some capacity.

Code		**Other**	
Git-Hub / Tools Set up	Divya	Project Proposal	Sophia
Data Collection	Divya	Meeting Scheduling	Sophia
Data Cleaning	Divya	Standup 1	Sophia
EDA Visualizations	Jordan	Standup 2	Sophia
Feature Engineering	Sophia	Final Report	All
Part A Linear Model	Jordan		
Part A Tree Based Models	Divya		
Part A Failure Analysis	Divya		
Part B PCA	Sophia		
Part B KMeans	Sophia		
Part B Sensitivity Analysis	Sophia		

References

Related Works (GitHub Projects)

Sanghal, P. (n.d.). Inside-AirBNB-Dataset [Computer software]. GitHub. https://github.com/psanghal/Inside-AirBNB-Dataset

BlueWall UMich. (n.d.). airbnb-explorer [Computer software]. GitHub. https://github.com/bluewallumich/airbnb-explorer

Pratik94229. (n.d.). AIR-BNB-Dashboard-Tableau [Computer software]. GitHub. https://github.com/Pratik94229/AIR-BNB-Dashboard-Tableau

Data Sources

Inside Airbnb. (n.d.). Get the data. http://insideairbnb.com/get-the-data/

Los Angeles County Sheriff's Department. (2023). Part I and II crime data. https://lasd.org/transparency/part1and2crimedata/

City of Los Angeles. (2023). Crime data from 2020 to present. https://data.lacity.org/Public-Safety/Crime-Data-from-2020-to-Present/2nrs-mtv8

City and County of San Francisco. (2023). Police department incident reports: 2018 to present. https://data.sfgov.org/Public-Safety/Police-Department-Incident-Reports-2018-to-Present/wg3w-h783

City of San Diego. (2023). Police NIBRS crime data. https://data.sandiego.gov/datasets/police-nibrs/

City of Oakland. (2023). CrimeWatch crime data. https://data.oaklandca.gov/Public-Safety/CrimeWatch-Maps-Past-90-Days/ym6k-rx7a

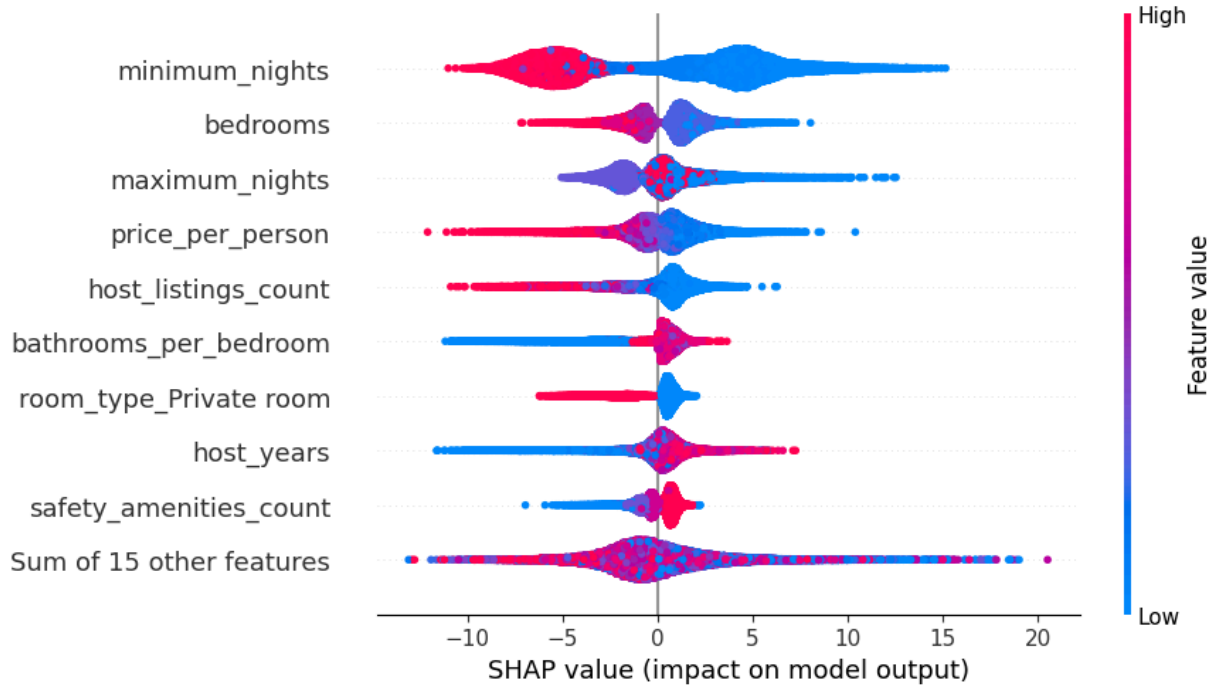
U.S. Census Bureau. (2023). American Community Survey 5-year estimates: Total population (Table B01003). https://data.census.gov

U.S. Environmental Protection Agency. (2021). Smart Location Database (Version 3). https://edg.epa.gov/EPADataCommons/public/OA/EPA_SmartLocationDatabase_V3_Jan_2021_Final.csv

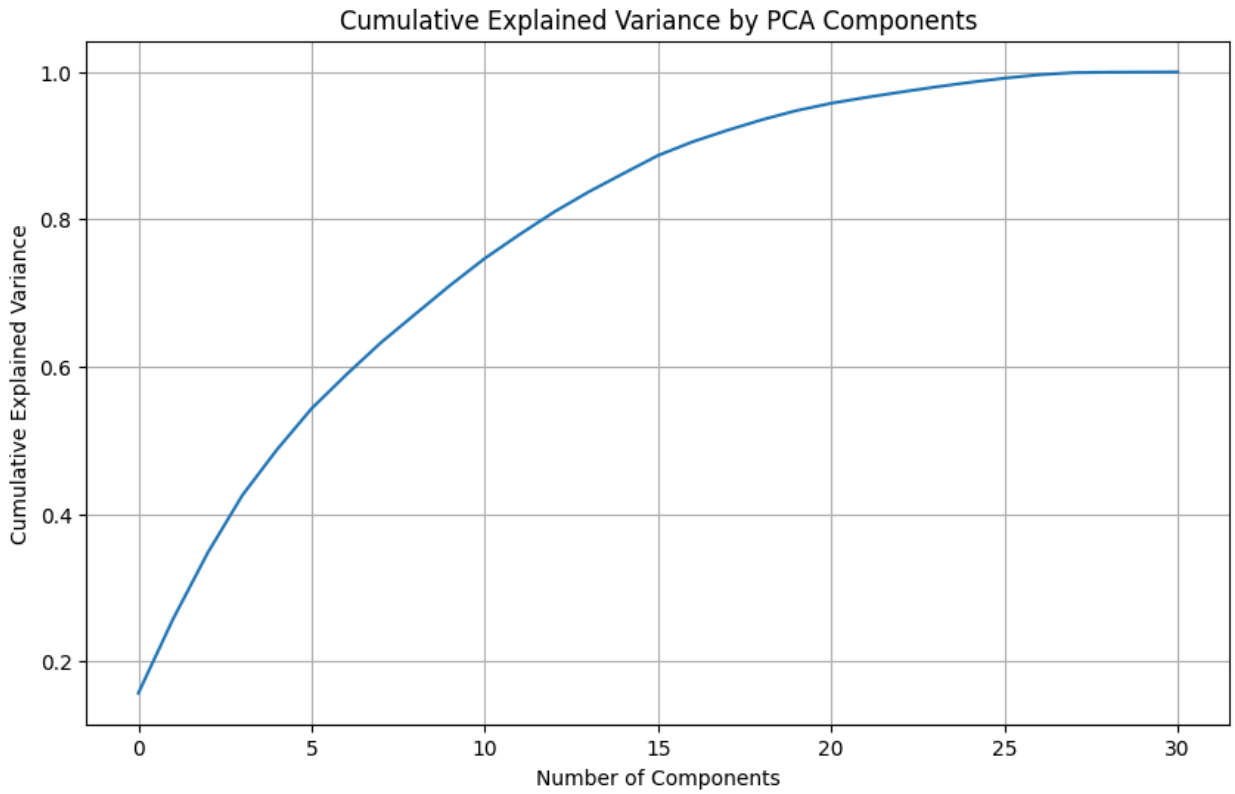
Technical Resource

Rey, S. J., & Arribas-Bel, D. (n.d.). Spatial autocorrelation. In Geographic Data Science with Python. https://geographicdata.science/book/notebooks/06_spatial_autocorrelation.html

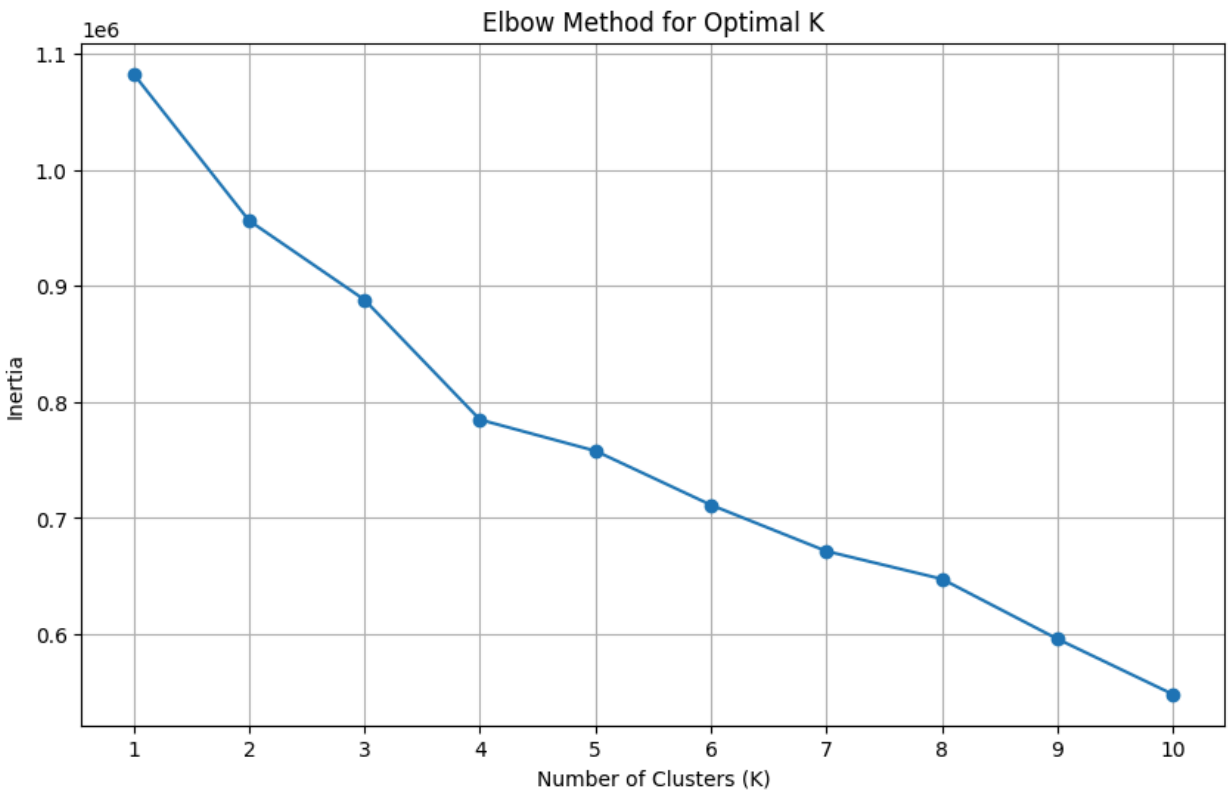
Appendix



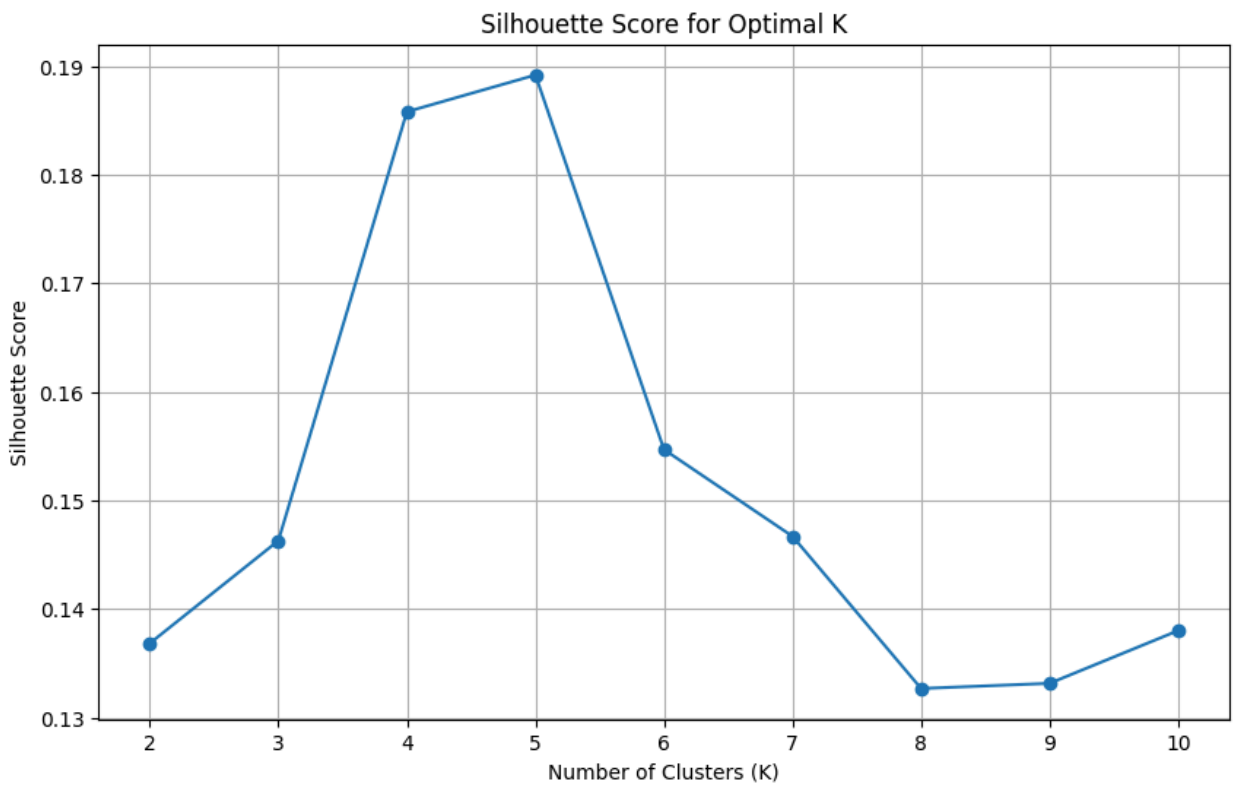
Appendix A-1



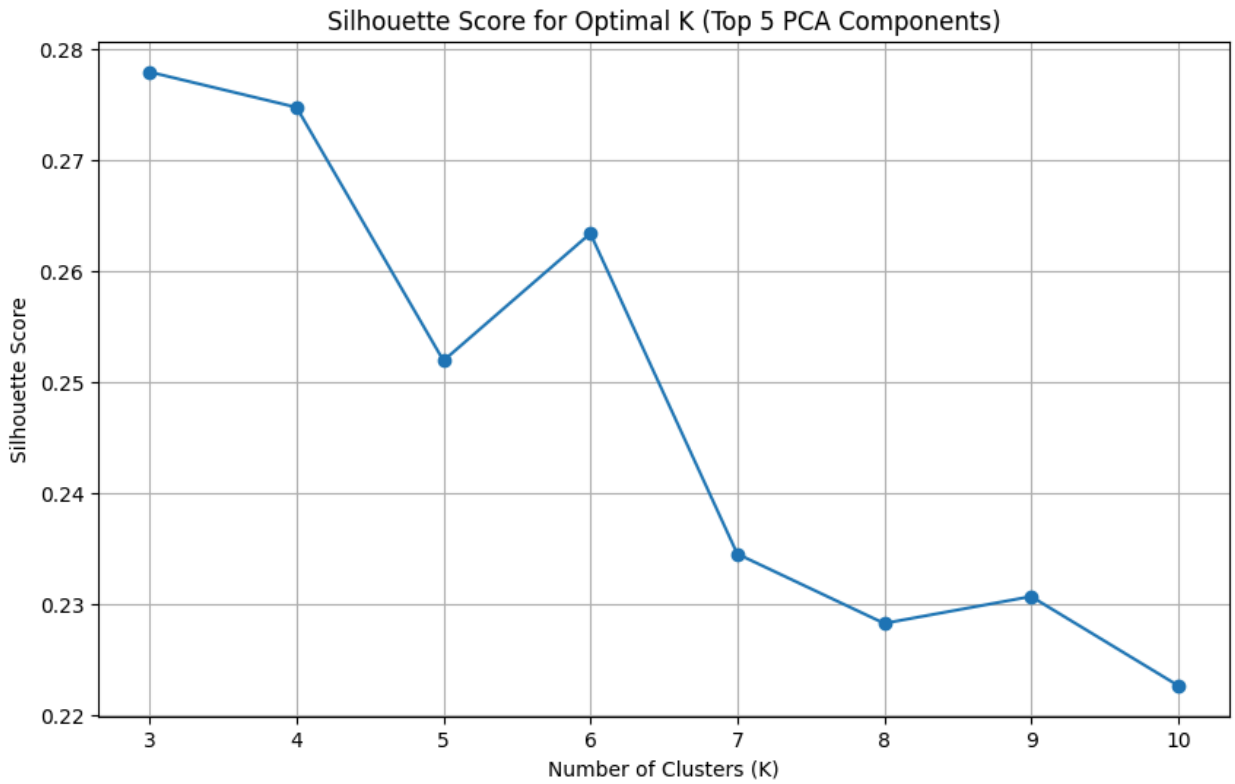
Appendix B-1



Appendix B-2



Appendix B-3


Silhouette Score for Optimal K (Top 5 PCA Components)
Silhouette Score
Number of Clusters (K)
0.28
0.27
0.26
0.25
0.24
0.23
0.22
3
4
5
6
7
8
9
10

Appendix B-4